

AI-Operation-Kaizen — Frontend Codebase Documentation

Project: AI-Operation-Kaizen

Branch: Anish-Frontend-kaizen

Platform: React Native (Expo)

Routing: Expo Router (File-based)

Location: FullStack/frontend/

Table of Contents

1. Project Overview

2. Root-Level Configuration Files

3. app/ — Screens & Navigation

- 3.1 Root Files
- 3.2 (auth)/ — Authentication Screens
- 3.3 (main)/ — Authenticated App Screens
- 3.4 (tabs)/ — Tab Navigation
- 3.5 dashboard/ — Dashboard Screens
- 3.6 issues/ — Issues Tab Screens

4. src/ — Core Source Library

- 4.1 components/chat/
- 4.2 components/common/
- 4.3 components/dashboard/
- 4.4 components/issue/
- 4.5 components/modals/
- 4.6 components/skeletons/
- 4.7 hooks/
- 4.8 services/
- 4.9 store/ & store/slices/
- 4.10 theme/
- 4.11 utils/
- 4.12 mocks/

5. assets/

6. scripts/

7. Complete Folder Structure at a Glance

1. Project Overview

AI-Operation-Kaizen is a React Native mobile application built with Expo and navigated using Expo Router — a file-system-based routing library. Every file inside the `app/` folder automatically maps to a route/screen in the application.

The frontend is entirely located under `FullStack/frontend/`. It follows a clean, modular architecture that separates responsibilities into distinct layers:

- **Screens** (`app/`) — All navigable pages of the app
- **Components** (`src/components/`) — Reusable UI building blocks
- **Services** (`src/services/`) — All API communication with the backend
- **State Management** (`src/store/`) — Global app state using Redux slices
- **Hooks** (`src/hooks/`) — Reusable logic shared across screens
- **Theme** (`src/theme/`) — Design tokens: colors, typography
- **Utils** (`src/utils/`) — Helper functions and utilities
- **Mocks** (`src/mocks/`) — Static test/development data

The app handles multiple user roles (field workers, supervisors, solvers, managers) and covers the full lifecycle of operational issues: submission, assignment, escalation, resolution, and review.

2. Root-Level Configuration Files

These files sit directly inside `FullStack/frontend/` and govern the project's configuration, tooling, and deployment pipeline.

`app.json`

The Expo application configuration file. Defines the app's name, slug (URL-safe identifier), app icon, splash screen image, supported orientations, and platform-specific settings for iOS and Android. This is the primary config file that Expo reads when building or launching the app.

`eas.json`

Expo Application Services (EAS) build configuration. Defines build profiles for different environments:

- **development** — For local development with debug tools enabled

- **preview** — For internal testing and QA
- **production** — For final app store submission

This file controls how the app gets compiled and packaged in the cloud via Expo's EAS Build service.

`metro.config.js`

Metro bundler configuration. Metro is React Native's JavaScript bundler (equivalent to Webpack for web). This config file customizes how modules are resolved, how assets are transformed, and any special file type handling needed for the project. It ensures the app bundles correctly for all target platforms.

`eslint.config.js`

ESLint code quality configuration. Defines linting rules to enforce consistent coding standards across the entire frontend codebase — things like unused variables, import ordering, and React-specific best practices. Helps catch bugs and maintain clean code.

`tsconfig.json`

TypeScript compiler configuration. Even though the project primarily uses `.js` files (not `.ts`), this file is present to set up path aliases and enable IDE-level type checking and autocompletion through JSDoc. It tells the TypeScript language server how to understand the project's module structure.

`vercel.json`

Vercel deployment configuration. Used when deploying the web-compatible version of the Expo app to Vercel. Configures routing rules (e.g., all paths point to the index for SPA routing) and any deployment-specific settings.

`package.json`

The **Node.js project manifest**. Lists all npm dependencies (React Native, Expo SDK, navigation libraries, Redux, etc.), dev dependencies (linters, testing tools), and defines scripts:

- `start` — Launches the Expo development server
- `android / ios / web` — Runs the app on a specific platform
- `build` — Triggers production build

`package-lock.json`

Auto-generated dependency lock file. Pins every package and its sub-dependency to exact version numbers, ensuring that all developers and CI/CD pipelines install the exact same dependency tree every time. This file should never be edited manually.

`README.md`

Project introduction and setup guide. Contains instructions for cloning the repo, installing dependencies, running the app locally, and understanding the basic project structure.

`vercel.json`

Vercel deployment configuration that handles SPA routing — ensures that direct URL access to any route redirects to the Expo web entry point rather than returning a 404.

3. `app/` — Screens & Navigation

The `app/` directory is the heart of Expo Router's file-system navigation. Each file represents a screen and each folder (especially those wrapped in parentheses like `(auth)` or `(main)`) represents a route group — a logical grouping of screens that share a layout but whose folder name does not appear in the URL/path.

3.1 Root Files

`app/index.js`

The **entry point** of the application. When the app launches, this is the first screen that evaluates the app's state. It checks:

1. Has the user completed onboarding before?
2. Is there a valid authentication token stored locally?

Based on this logic, it redirects the user to either the onboarding screen, the login screen, or the main app dashboard. It renders no visible UI of its own — it's purely a redirect controller.

`app/_layout.js`

The **root layout component** for the entire application. In Expo Router, a `_layout.js` at the root level wraps every other screen in the app. This file is responsible for:

- Setting up the **Redux store provider** so all screens have access to global state
- Setting up the **Theme context** so all screens can access colors and typography
- Configuring the **navigation container** and **stack navigator**

- Handling authentication guards — checking if the user is logged in and redirecting them appropriately
- Loading any fonts or global assets needed before the app renders

This is the most critical single file in the frontend — everything flows through it.

`app/onboarding.js`

The onboarding experience shown to first-time users of the app. This is a substantial file (~30KB) featuring:

- A multi-step slideshow that walks the user through what the app does
- Animated transitions between slides
- Progress indicators showing which step the user is on
- A "Get Started" button on the last slide that marks onboarding as complete and routes the user to login
- The onboarding state is persisted locally so returning users are never shown it again

3.2 `(auth)/` — Authentication Screens

Route group for all login and authentication-related screens. The `(auth)` wrapper means these screens share a layout but `auth` does not appear in the navigation path. Users who are already authenticated cannot navigate back into this group.

`(auth)/_layout.js`

Defines the stack navigator layout for the auth screens. Ensures proper back-navigation behavior and prevents authenticated users from accessing login screens again. Sets up any shared headers or animations for the auth flow.

`(auth)/login.js`

The Login screen — the primary authentication entry point. Features:

- Email and password input fields with real-time validation
- Error message display for incorrect credentials or server errors
- A "Login" button that triggers the authentication API call
- On success: stores the auth token and navigates to the main app
- On failure: displays a user-friendly error message
- Integration with the `authSlice` Redux store to persist the user's session

3.3 (main)/ — Authenticated App Screens

Route group that wraps all screens only accessible to logged-in users. Any attempt to access these routes without authentication redirects back to login.

(main)/_layout.js

The **authenticated area layout wrapper**. Verifies the user's auth token on mount and sets up the navigation structure for the entire authenticated portion of the app, including any shared headers or navigation options.

(main)/profile.js

The **User Profile screen**— accessible from the navigation header or a dedicated profile button. This is a feature-rich screen (~11KB) that displays:

- The logged-in user's **name, role, and profile photo** (or avatar initials)
- **Assigned sites** the user is responsible for
- **Account details** and contact information
- **Activity summary** showing recent actions
- A **Logout button** that clears the auth token and redirects to login
- Any role-specific information (e.g., supervisor's site list, solver's skill tags)

3.4 (tabs)/ — Tab Navigation

Nested group that defines the **bottom tab bar** — the primary navigation structure visible to authenticated users at all times.

(tabs)/_layout.js

Configures the **bottom tab bar** component. Defines:

- Which screens appear as tabs (Dashboard and Chat)
- Icons for each tab (active and inactive states)
- Labels shown below each icon
- Tab bar colors and styling (background, active indicator)
- Any badge indicators for notifications

`(tabs)/chat.js`

The AI Chat screen — one of the two main tabs. This is the core AI interaction surface of the app (~15KB). It:

- Renders the full **conversation interface** with a scrollable message history
- Displays the AI responses and user messages in styled bubbles
- Integrates `ChatInput.js`, `ChatMessage.js`, `ChatHistorySidebar.js`, and `NotificationBanner.js` components
- Manages **chat session state** — current session, active session ID
- Connects to the `chatSlice` Redux store for message history
- Handles the **sidebar toggle** for accessing past conversations
- Manages the **keyboard avoidance** behavior so the input bar stays visible when typing

3.5 `dashboard/` — Dashboard Screens

This is the largest section of the app — a complete operational management hub nested inside the tabs navigator. It contains 17 screens covering every aspect of the issue management workflow.

`dashboard/_layout.js`

The **stack navigator** layout for all dashboard screens. Manages animated screen transitions when navigating from list views to detail views and controls the back-navigation header.

`dashboard/index.js`

The **main Dashboard home screen** and the app's primary landing page after login. This is the largest screen file (~27KB), reflecting its complexity:

- **Summary metric cards** showing counts of: open issues, escalated issues, fixed issues, complaints, and pending reviews
- **Quick-access navigation** to each category (Issues, Complaints, Sites, Solvers, Escalated, Fixed, Not Fixed, Awaiting Review)
- **Charts and trend indicators** showing operational health over time
- **Recent activity feed** listing the latest changes across the system
- **Pull-to-refresh** functionality to get the latest data
- Uses `DashboardCard.js` components for each metric tile

dashboard/issues.js

The Issues list screen from the dashboard perspective (manager/supervisor view). Displays:

- A filterable, searchable list of all operational issues across all sites
- Each item shows: issue title, assigned site, assigned solver, priority level, and current status badge
- Filter and sort options (by status, site, date, priority)
- Tapping an issue navigates to dashboard/issue-detail.js

dashboard/issue-detail.js

The comprehensive detail view for a single issue in the dashboard context (~27KB). This is a full management view showing:

- Complete issue description, category, and priority
- Assigned solver information with a link to their profile
- Issue photos via the ImageGallery component
- GPS location on a map
- Status history timeline showing every status change
- Action buttons for managers: reassign, escalate, close, or send back
- Comment thread for notes and updates

dashboard/complaints.js

Lists all user-submitted complaints in the system. Each complaint card shows:

- The complaint title and description excerpt
- The submitter's name and submission date
- Current status (Open, In Review, Resolved)
- Filtering by status and site

dashboard/complaint-detail.js

The full detail view of a single complaint (~15KB). Displays:

- The complete complaint text submitted by the user
- The associated site and location
- Resolution notes added by the handling supervisor
- Timestamps for submission and last update
- Action options: mark as reviewed, add a note, escalate, or resolve

`dashboard/escalated.js`

Lists all **escalated issues** — issues that have been flagged as requiring urgent senior attention. Provides:

- A high-priority view for supervisors and managers
- Visual indicators for urgency level and time since escalation
- Quick-navigation to each escalated issue's detail

`dashboard/escalated-detail.js`

The **detail screen for an escalated issue** — the most feature-complete detail screen in the dashboard (~26KB). Includes:

- Full issue information and escalation reason
- **Complete audit trail**: who escalated it, when, and why
- **Escalation history** showing all previous escalation attempts
- Options to reassign to a senior solver, de-escalate, or mark as resolved
- Communication thread between supervisors and solvers

`dashboard/fixed.js`

Lists all **resolved/fixed issues** — the historical record of completed work. Features:

- A searchable archive of completed issues
- Each item shows: resolution date, time-to-fix, assigned solver, and site
- Filterable by site, solver, and date range

`dashboard/fixed-detail.js`

The **detail view for a fixed issue** (~14KB). Shows:

- Full issue information with original report details
- The **resolution note** written by the solver
- **Before/after photos** if submitted
- Time taken to resolve
- The solver who completed the work

dashboard/not-fixed.js

Lists all issues marked as "Not Fixed" — cases where a resolution was attempted but the problem persists. These require follow-up action. Shows:

- The original issue details
- Previous resolution attempts
- Current status and urgency level

dashboard/not-fixed-detail.js

The **detail view** for a **not-fixed** issue (~24KB). Includes:

- Full issue history including all previous fix attempts
- Notes from the solver explaining why it couldn't be resolved
- Options for the manager: reassign to a different solver, escalate, or request more information
- A timeline of all actions taken on the issue

dashboard/awaiting_review.js

Lists all issues in "Awaiting Review" status — the solver has submitted their work and is waiting for managerial sign-off before the issue is officially closed. Shows:

- Issues pending approval with submission timestamps
- The solver who submitted the resolution
- Quick navigation to review each one

dashboard/awaiting_review_detail.js

The **detail view** for an issue awaiting review (~18KB). The manager's sign-off screen. Includes:

- The solver's resolution submission with notes and photos
- A side-by-side view of the original report vs. the resolution
- **Approve button** — marks the issue as Fixed and closes it
- **Reject button** — sends it back to the solver with remarks
- A remarks/feedback field for the manager to explain their decision

dashboard/sites.js

Lists all **operational sites** (physical locations/facilities) managed within the system (~13KB). Each site card shows:

- Site name and address
- Open issue count
- Assigned solver count
- Overall operational health indicator
- Filterable and searchable by name or region

dashboard/site-detail.js

The **full detail view for a single site** (~13KB). A site-centric management screen showing:

- Site information: name, address, contact details
- All **open issues** at this site
- All **assigned solvers** for this site
- Recent complaint history
- A link to view escalated issues at this site

dashboard/solvers.js

Lists all **solvers** (field technicians/maintenance engineers) registered in the system (~14KB). Each solver card shows:

- Name and profile photo/avatar
- Assigned sites
- Current active issue count (workload indicator)
- Performance rating
- Filterable by site, status, and availability

dashboard/solver-profile.js

The **detailed profile view of a single solver** (~14KB). A comprehensive performance dashboard showing:

- Solver's personal information and contact details
- **Performance metrics**: total issues resolved, average time-to-fix, escalation rate
- List of **currently active issues** assigned to them
- **Issue history** — all past resolved, not-fixed, and escalated issues

- Skill tags and specializations
- Site assignments

3.6 `issues/` — Issues Tab Screens

A dedicated section under the tabs navigator for the **field worker and reporter perspective** on issues — as opposed to the manager-centric dashboard view.

`issues/_layout.js`

Stack navigator layout for the issues tab — manages navigation transitions between the issues list and detail screens.

`issues/index.js`

The Issues list screen from a field worker's perspective (~20KB). This is the primary workspace for users who report and track issues. Features:

- A comprehensive list of all issues the user has access to
- **Advanced filtering** by status (Open, Escalated, Fixed, Not Fixed, Awaiting Review), site, priority, and date
- Search by issue title or description
- A "Report New Issue" button that opens the issue creation flow
- Pull-to-refresh to fetch latest data
- Uses `IssueCard.js` for each list item and `IssueCardSkeleton.js` during loading

`issues/issue-detail.js`

The full issue detail view — the largest single screen file in the entire project (~31KB). This is a role-adaptive screen that renders differently depending on the logged-in user's role (reporter, solver, supervisor). It includes:

- **Complete issue information:** title, description, category, priority, and site
- **Status badge** showing current state in the workflow
- **ImageGallery** showing all attached photos
- **GPS location map** showing where the issue was reported
- **IssueTimeline** component showing the full history of status changes and actions
- **CallHistorySection** showing any calls made related to this issue
- **Solver assignment** information and a link to the solver's profile
- **Role-based action buttons:**

- Reporters can: add a comment, add more photos
- Solvers can: mark as Fixed, mark as Not Fixed, request more info
- Supervisors can: escalate, reassign, close, approve or reject submissions
- **Comment/activity thread** for team communication on the issue

4. `src/` — Core Source Library

The `src/` folder contains all shared building blocks used across screens — reusable components, global state, API services, hooks, theming, and utility functions.

4.1 `components/chat/`

Components that power the AI Chat tab screen.

`ChatHistorySidebar.js`

A **slide-in sidebar panel** for the chat screen. When the user taps the history button, this sidebar slides in from the side and displays:

- A list of all **past chat sessions** with timestamps and preview text
- "New Chat" button to start a fresh conversation
- Ability to **switch between sessions** — tapping a session loads that conversation in the main chat view
- Session deletion option

`ChatInput.js`

The **message input bar** at the bottom of the chat screen. This is the most complex chat component (~27KB) handling multiple input modalities:

- **Text input field** that auto-grows with content
- **Send button** that becomes active when there's text or a file
- **Voice input trigger** — activates microphone for speech-to-text
- **Attachment handling** — allows attaching images or files to the message
- **Keyboard avoidance** — the input bar floats above the keyboard when it opens
- **Input state management**: loading indicator while waiting for AI response, disabled state while processing

`ChatMessage.js`

Renders a single message bubble in the conversation. Differentiates visually between:

- User messages: right-aligned, colored bubble
- AI responses: left-aligned, white/grey bubble with the app's AI avatar
- Supports markdown-style formatting in AI responses (bold, lists, code blocks)
- Shows timestamp on long-press or below the message
- Handles loading/typing indicator while the AI is generating a response

`NotificationBanner.js`

A contextual alert banner that appears within the chat interface. Used to display:

- Important system alerts without navigating away from the chat
- Issue status update notifications
- New assignment notifications
- Banners are dismissible and auto-timeout after a configurable duration

4.2 components/common/

The shared UI primitive library — reusable building blocks used across all screens in the app.

`AnimatedButton.js`

A button component with built-in press animations. When pressed, it scales down slightly and reduces opacity, giving tactile visual feedback. Used anywhere a more polished, interactive feel is needed beyond the standard Button. Wraps React Native's `Animated` API.

`Avatar.js`

Renders a user profile avatar. Shows the user's profile photo if available, or falls back to a colored circle with their initials if no photo exists. Supports multiple size variants (small, medium, large) for use in list items, headers, and profile pages.

`Button.js`

The standard button component used throughout the app. Supports:

- Variants: primary (filled), secondary (outlined), danger (red)
- Loading state: shows a spinner inside the button while an action is in progress
- Disabled state: greys out and ignores presses

- Full-width or auto-width layout modes

`Card.js`

A generic card container with a white background, rounded corners, and a subtle drop shadow. Used to visually group related content in list items, dashboard tiles, and detail sections. Accepts any child content.

`EmptyState.js`

Displayed when a list or data section has no items. Shows:

- A relevant icon
- A friendly descriptive message (e.g., "No issues found")
- Optionally, a call-to-action button (e.g., "Report an Issue")

`ErrorState.js`

Displayed when an API call fails or data cannot be loaded. Shows:

- An error icon
- A brief error message
- A "Try Again" / Retry button that re-triggers the failed fetch

`FullScreenSpinner.js`

A full-screen loading overlay with a centered activity spinner. Used during critical operations where the user should not interact with any other part of the app — for example, during initial app loading or authentication.

`Input.js`

The styled text input component used in forms throughout the app. Features:

- A label above the field
- Consistent styling with the app theme
- Error message display below the field for validation feedback
- Secure text mode (for passwords) with an eye icon toggle
- Placeholder text support

`Loader.js`

A **lightweight inline loading spinner** for smaller, contained loading states within a screen — such as while a sub-section of data loads. Unlike `FullScreenSpinner.js`, this does not take over the whole screen.

`LocationCapture.js`

A **rich GPS location capture component** (~11KB). Used when reporting a new issue. Features:

- Requests the device's **location permission**
- Fetches the user's **current GPS coordinates**
- Displays a **map preview** showing the captured location
- Allows the user to **manually drag a pin** to adjust the location if needed
- Shows the resolved **street address** alongside the coordinates

`OfflineBanner.js`

A **persistent connectivity banner** that appears at the top of the screen whenever the device loses internet access. It:

- Monitors the device's **network state** in real time
- Displays a clearly visible **"You are offline"** message
- Automatically dismisses itself when connectivity is restored

`PhotoUploader.js`

Handles **photo capture and selection** for issue reporting (~6KB). Lets the user:

- Take a new **photo** using the device camera
- Choose from the gallery to attach an existing image
- See a **thumbnail preview** of the selected photo
- Remove the photo before submitting
- Prepares the image data in the correct format for upload

`Shimmer.js`

The **base shimmer animation primitive**. A single animated pulsing shape (rectangle, circle, etc.) that simulates a loading placeholder. This is the building block used by `SkeletonLoaders.js` and the skeleton components to build full screen-shaped loading states.

`SkeletonLoaders.js`

Assembles `Shimmer.js` primitives into **full structured loading placeholders** that match the shape of actual screen content. For example, a "card skeleton" looks like a real card but with grey pulsing rectangles instead of real text and images. This is shown while data is being fetched to prevent layout shift and improve perceived performance.

`StatusBadge.js`

A **colored pill/badge component** that displays an issue or complaint's current status. Each status maps to a distinct color:

- Open → Blue
- Escalated → Red/Orange
- Fixed → Green
- Not Fixed → Red
- Awaiting Review → Yellow
- Closed → Grey

`Toast.js`

A **temporary pop-up notification** that slides in from the bottom of the screen. Used for brief feedback messages:

- Success confirmations (e.g., "Issue reported successfully")
- Error alerts (e.g., "Failed to update status")
- Info messages (e.g., "Syncing data...")
- Auto-dismisses after a set duration

4.3 `components/dashboard/`

Components specifically designed for the Dashboard screens.

`DashboardCard.js`

A **metric summary tile** used on the main dashboard index screen. Displays:

- A **large number** representing a count (e.g., 12 open issues)
- A **label** describing what the number represents
- A **colored icon** for visual differentiation
- Tap navigation to the corresponding list screen (e.g., tapping the Issues card goes to `dashboard/issues.js`)

`ChartDownloadButton.js`

A **button component** that allows the user to **download or export a chart** displayed on the dashboard as an image or PDF. Integrates with the `pdfExport.js` utility to generate the downloadable file.

4.4 `components/issue/`

Components dedicated to displaying issue-related data in detail views.

`IssueCard.js`

The **list item card component** for displaying a single issue in a list (~11KB). Shows:

- Issue title and a brief description excerpt
- **StatusBadge** for the current status
- Site name and location
- Priority indicator (color-coded)
- Assigned solver's name or "Unassigned"
- Submission date/time
- Tap to navigate to the issue's detail screen

`IssueTimeline.js`

A **vertical timeline component** that renders the complete history of an issue. Each timeline event shows:

- The **action taken** (e.g., "Issue Reported", "Assigned to John", "Escalated", "Marked Fixed")
- The **actor** who performed the action (user name and role)
- The **timestamp** of the action
- An icon appropriate to the event type

`ImageGallery.js`

A **horizontal scrollable photo gallery** for displaying all images attached to an issue. Features:

- Thumbnail strip showing all photos
- Tap a thumbnail to **view the full-size image** in a modal
- Pinch-to-zoom on the full-size image
- Photo count indicator (e.g., "3 photos")

`CallHistorySection.js`

Displays a log of phone calls related to a specific issue. Shows:

- Caller and recipient names
- Call direction (inbound/outbound)
- Call timestamp and duration
- Used to track communication history between supervisors, solvers, and reporters regarding a specific issue

4.5 `components/modals/`

Overlay modal components used across the app.

`FilterModal.js`

A **bottom-sheet modal** (~15KB) for applying filters and sorting options to list screens (issues, complaints, sites, solvers). Users can:

- Select **status filters** (multi-select, e.g., Open + Escalated)
- Filter by site
- Filter by date range
- Choose **sort order** (newest first, oldest first, priority)
- Apply all filters at once or reset to defaults
- Used by the Issues list, dashboard lists, and complaints list

4.6 `components/skeletons/`

Screen-specific skeleton loading placeholders.

`DashboardCardSkeleton.js`

A **skeleton placeholder** that mimics the shape and layout of a `DashboardCard` on the main dashboard. Shown while the dashboard metrics are being fetched from the API. Multiple instances are stacked to fill the screen during initial load.

`IssueCardSkeleton.js`

A **skeleton placeholder** matching the layout of an `IssueCard` list item. Shown in the issues list while the issue data is loading. Gives the user a sense of the expected content structure before real data arrives.

4.7 hooks/

Custom React hooks that encapsulate reusable stateful logic.

`useDebounce.js`

A utility hook that **delays the processing of a rapidly changing value** until the user pauses. Used primarily in search inputs — when the user types in the search bar, this hook waits until they stop typing for a short period (e.g., 300ms) before triggering an API call. Prevents spamming the server with a request for every single keystroke.

`useNetworkStatus.js`

A hook that **monitors the device's internet connectivity** in real time. Returns a boolean `isConnected` value that any component can subscribe to. Powers the `OfflineBanner.js` component and the offline state in the Redux store. Uses the `network.js` utility under the hood.

`useSmartBack.js`

A hook that handles **intelligent back navigation behavior**. In certain screens, pressing the hardware back button (Android) should confirm with the user before leaving (e.g., "You have unsaved changes, are you sure you want to go back?"). This hook manages that logic so it doesn't need to be re-implemented in each screen individually.

4.8 services/

All communication with the backend API is centralized here. Screens never directly make HTTP requests — they call functions from these service files.

`api.js`

The **primary API service file** (~24KB) — the largest and most important file in `services/`. It contains:

- An **Axios instance** configured with the backend base URL and default headers
- **Request interceptors** that automatically attach the authentication token to every request
- **Response interceptors** that handle 401 (unauthorized) errors globally by logging the user out
- **Named API functions** covering all backend endpoints:
 - Authentication (login, logout, token refresh)
 - Issues (fetch all, fetch by ID, create, update status, assign solver)
 - Complaints (fetch all, fetch by ID, update)

- Dashboard metrics (summary counts, charts data)
- Sites (fetch all, fetch by ID)
- Solvers (fetch all, fetch profile, fetch performance)
- Chat (send message, fetch history, start new session)
- User profile (fetch, update)

`imagekitService.js`

A dedicated service for image uploads using the ImageKit media management platform (~5KB). When a user attaches photos to an issue report, this service:

- Fetches a **temporary upload authentication token** from the backend
- Uploads the image directly to ImageKit's CDN
- Returns the **hosted image URL** which is then stored with the issue record
- Handles upload progress and error states

4.9 `store/` & `store/slices/`

Global state management using **Redux Toolkit**. The store is the single source of truth for all app-wide data.

`store/index.js`

The **Redux store configuration**. Combines all slice reducers into a single store and applies any middleware (e.g., for async operations with Redux Thunk). This is the store instance provided to the entire app via the root layout.

`store/slices/authSlice.js`

Manages **authentication state** globally. Holds:

- The logged-in user object (name, role, ID, assigned sites)
- The auth token
- Login, logout, and token refresh actions
- Persists the token to secure local storage so the user stays logged in across app restarts

`store/slices/chatSlice.js`

Manages the **AI chat state**. Holds:

- All **chat sessions** and their message histories
- The **currently active session ID**
- Loading state for when a message is being processed by the AI
- Actions to add messages, start new sessions, and switch sessions

`store/slices/complaintsSlice.js`

Manages **complaints data** globally. Holds:

- The list of all complaints
- Loading and error states for complaints fetching
- Actions to fetch complaints, update a complaint's status, and cache the results

`store/slices/dashboardSlice.js`

Manages **dashboard metrics and summary data**. Holds:

- Cached counts for issues, complaints, escalated, fixed, not-fixed, and awaiting review
- Loading state for the dashboard
- Actions to refresh dashboard data

`store/slices/issuesSlice.js`

The **largest and most complex slice** (~17KB). Manages all issues data:

- Full list of all issues with pagination support
- Individual issue details cache
- Filter state (current active filters)
- Loading and error states for all issue operations
- Actions for: fetching issues, fetching a single issue, creating an issue, updating status, assigning a solver, adding a comment

`store/slices/notificationsSlice.js`

Manages **notification state**. Holds:

- Count of unread notifications
- Actions to mark notifications as read

- Drives the badge indicator on tab bar icons

`store/slices/offlineSlice.js`

Manages **offline/connectivity** state. Holds:

- The current network connectivity status
- A queue of **pending actions** to be executed when connectivity is restored
- Actions to add to the queue and process the queue on reconnect

`store/slices/performanceSlice.js`

Manages **solver performance data**. Holds:

- Performance metrics for solvers (resolution rates, average time-to-fix)
- Actions to fetch and cache performance data

`store/slices/sitesSlice.js`

Manages **sites data**. Holds:

- The list of all sites
- Actions to fetch sites and cache the results
- Used by filter dropdowns and site-detail screens

`store/slices/themeSlice.js`

Manages the **active theme** (light/dark mode). Holds:

- Current theme mode preference
- Action to toggle between light and dark
- Persisted to local storage so the user's preference is remembered

4.10 theme/

The design system definition for the entire app.

`colors.js`

Defines the complete color palette of the application. Contains:

- Primary and secondary brand colors
- Status colors (success green, danger red, warning yellow, info blue)
- Neutral grays for backgrounds, borders, and text
- Both light mode and dark mode color sets
- All components and screens import from this file rather than using raw hex codes — ensuring visual consistency

`typography.js`

Defines the text style system:

- Font size scale (xs, sm, base, lg, xl, 2xl, etc.)
- Font weight definitions (regular, medium, semibold, bold)
- Line height values
- Letter spacing
- Pre-built text style presets (e.g., `heading1`, `body`, `caption`)

`ThemeContext.js`

A **React Context** that makes the current theme (colors, typography) available to any component in the app without manually passing it as props. The root layout wraps the app with this context provider, and any component can call `useTheme()` to access the design tokens.

4.11 `utils/`

Pure helper functions and platform utilities used across the codebase.

`animations.js`

A collection of reusable animation presets and helpers (~5KB) built on React Native's `Animated` API. Contains:

- Fade in/out animation configurations
- Slide up/down presets
- Scale pulse animations
- Helper functions to create and start standard animations without boilerplate

constants.js

App-wide constant values that are referenced in multiple places:

- API status codes and their meanings
- Issue status strings (e.g., "open", "escalated", "fixed")
- Priority levels
- Pagination size defaults
- Timeout durations

formatters.js

Data formatting helper functions:

- Date and time formatting (e.g., "2 hours ago", "Apr 10, 2026")
- Phone number formatting
- Name truncation for display in constrained spaces
- Issue status label formatting

haptics.js

Haptic feedback utilities (~5KB) for triggering device vibration feedback on user interactions:

- Light tap for button presses
- Medium impact for confirmations
- Heavy impact for errors or warnings
- Notification patterns for success/error alerts
- Gracefully degrades on devices that don't support haptics

location.js

Basic GPS location helper functions used across the app:

- Request location permissions
- Get current coordinates
- Convert GPS coordinates to a human-readable address (reverse geocoding)

`locationCapture.js`

An **advanced location capture utility** (~7KB) with more robust logic for the issue-reporting flow:

- Handles permission request flow with user-friendly prompts
- Implements retry logic if location access times out
- Formats location data for API submission
- Used by the `LocationCapture.js` component

`network.js`

Core **network status monitoring** utility. Uses `@react-native-community/netinfo` to detect connectivity changes and exports helper functions consumed by the `useNetworkStatus` hook and the `offlineSlice`.

`networkRetry.js`

A **smart retry utility** (~4KB) for failed API calls. Implements:

- Exponential backoff retry logic (retries after 1s, 2s, 4s, etc.)
- Configurable maximum retry attempts
- Distinguishes between retryable errors (network timeouts) and non-retryable errors (401 auth errors)

`notificationNavigation.js`

Handles **deep-link navigation from push notifications** (~5KB). When the user taps a push notification while the app is in the background or closed, this utility:

- Parses the notification payload to determine the destination screen
- Navigates the user directly to the relevant issue, complaint, or chat session
- Works with Expo Notifications for notification delivery

`overdue.js`

Overdue detection utilities for issues. Contains functions that:

- Calculate whether an issue has exceeded its expected resolution time
- Return a human-readable overdue label (e.g., "Overdue by 3 hours")
- Used by `IssueCard.js` and detail screens to highlight overdue items

`pdfExport.js`

PDF generation utility (~3KB) that allows dashboard charts and issue details to be exported as PDF files directly from the app. Uses a PDF generation library to format the content and trigger a share/download dialog.

`permissions.js`

Device permission management utility. Contains helper functions to:

- Request camera permission (for photo upload)
- Request location permission (for issue reporting)
- Request notification permission (for push alerts)
- Check current permission status before attempting to use device features

`photoUpload.js`

Photo upload utility (~6KB) handling the full image upload pipeline:

- Compress the image before upload (to reduce bandwidth)
- Convert to the correct format
- Upload via `imagekitService.js`
- Return the hosted CDN URL
- Handle upload failure with user-friendly error messages

`scoreEngine.js`

Solver performance scoring utility (~4KB). Calculates performance metrics for solver profiles:

- Resolution rate (issues fixed / issues assigned)
- Average time-to-resolution
- Escalation rate
- Overall performance score/rating
- Used by the solver profile screen and the solvers list for ranking

`storage.js`

Local storage utility wrapping `AsyncStorage` (React Native's persistent key-value store). Provides simple, clean functions for:

- Saving and retrieving the auth token
- Saving user preferences

- Clearing stored data on logout

`voiceUtil.native.js`

The **native (iOS/Android) voice input utility**. Platform-specific implementation for triggering device speech recognition. The `.native.js` extension tells Metro bundler to use this file only on native platforms (iOS/Android), not on web.

`voiceUtils.js`

The **cross-platform voice utilities** file used by `ChatInput.js`. Contains:

- Start/stop recording functions
- Speech-to-text result handling
- Error handling for microphone permission denial
- Falls back gracefully if the device doesn't support voice input

4.12 `mocks/`

Static mock data files used during **development and testing** when the real API is not yet available or when testing specific UI states.

`apiService.js`

A **mock API service** that mirrors the real `services/api.js` but returns hardcoded data instead of making HTTP requests. Used in development to work on UI without needing the backend running.

`issues.js`

A large set of **hardcoded issue objects** (~12KB) representing many issues in various states (open, escalated, fixed, etc.) with realistic data. Used to develop and test the issues list and detail screens.

`complaints.js`

Mock complaint data — a set of sample complaints with different statuses and submitters.

`chatMessages.js`

Mock chat conversation data — sample messages between a user and the AI assistant for developing and testing the chat UI.

`callLogs.js`

Mock call history data — sample phone call logs used to develop and test the `CallHistorySection.js` component.

`issueAssignments.js`

Mock assignment data — sample records of issue-to-solver assignments used to develop and test assignment-related UI.

`issueHistory.js`

Mock issue status history data — sample timeline events used to develop and test the `IssueTimeline.js` component.

`issueImages.js`

Mock image attachment data — sample image URLs used to develop and test the `ImageGallery.js` component.

`sites.js`

Mock site data — a small set of sample operational sites.

`solverSkills.js`

Mock solver skill data — sample skill tags and specializations for solver profiles.

`supervisorSites.js`

Mock supervisor-to-site assignment data — sample data showing which sites each supervisor manages.

`users.js`

Mock user data — sample user accounts across different roles (reporter, solver, supervisor, admin) used to develop profile and user-related screens.

5. `assets/`

Static files bundled with the application.

`assets/fonts/`

`SpaceMono-Regular.ttf`

The **SpaceMono Regular** typeface file (~91KB). A monospaced font used for specific UI elements (e.g., code snippets, data labels, IDs). Bundled directly with the app so it renders consistently across all devices without relying on system fonts.

`assets/images/`

Contains all **static image assets** used in the app:

- App icon (used on the home screen)
- Splash screen image (shown at launch)
- Onboarding illustration images
- Placeholder images for empty states
- Any other static graphic assets

6. `scripts/`

Utility scripts for development and project management. These are Node.js or shell scripts run from the command line by developers — they are not part of the app itself.

Common purposes include:

- **Project reset** — clearing caches and resetting the Expo environment
- **Asset generation** — automatically generating different icon sizes for iOS and Android from a master image
- **Environment setup** — setting up environment variables for different deployment targets

7. Complete Folder Structure at a Glance

```
FullStack/frontend/
├─ app.json                ← Expo app config
├─ eas.json                ← EAS Build config
├─ metro.config.js         ← Bundler config
├─ eslint.config.js        ← Code linting rules
├─ tsconfig.json           ← TypeScript / IDE config
├─ vercel.json             ← Web deployment config
├─ package.json            ← Dependencies & scripts
├─ package-lock.json       ← Locked dependency versions
├─ README.md               ← Project setup guide
├─
├─ app/
│   ├── _layout.js         ← Root layout, providers, auth gate
│   ├── index.js           ← Entry point / redirect logic
│   ├── onboarding.js      ← First-time user onboarding (30KB)
│   ├──
│   ├── (auth)/
│   │   ├── _layout.js     ← Auth stack layout
│   │   └─ login.js        ← Login screen
│   ├──
│   ├── (main)/
│   │   ├── _layout.js     ← Authenticated area layout
│   │   └─ profile.js      ← User profile screen
│   ├──
│   ├── (tabs)/
│   │   ├── _layout.js     ← Bottom tab bar configuration
│   │   ├── chat.js        ← AI Chat tab (15KB)
│   │   ├──
│   │   ├── dashboard/
│   │   │   ├── _layout.js ← Dashboard stack layout
│   │   │   ├── index.js  ← Dashboard home / metrics (27KB)
│   │   │   ├── issues.js  ← All issues list
│   │   │   ├── issue-detail.js ← Issue detail (dashboard view, 27KB)
│   │   │   ├── complaints.js ← Complaints list
│   │   │   ├── complaint-detail.js ← Complaint detail
│   │   │   ├── escalated.js ← Escalated issues list
│   │   │   ├── escalated-detail.js ← Escalated detail (26KB)
│   │   │   ├── fixed.js   ← Fixed issues list
│   │   │   ├── fixed-detail.js ← Fixed issue detail
│   │   │   ├── not-fixed.js ← Not-fixed issues list
│   │   │   ├── not-fixed-detail.js ← Not-fixed detail (24KB)
│   │   │   ├── awaiting_review.js ← Awaiting review list
│   │   │   ├── awaiting_review_detail.js ← Review sign-off (18KB)
│   │   │   ├── sites.js   ← Sites list
│   │   │   ├── site-detail.js ← Site detail
│   │   │   ├── solvers.js ← Solvers list
│   │   │   └─ solver-profile.js ← Solver profile & performance
│   │   ├──
│   │   └─ issues/
│   │       ├── _layout.js ← Issues tab stack layout
│   │       ├── index.js  ← Issues list (field worker view, 20KB)
│   │       └─ issue-detail.js ← Full issue detail (31KB)
│   └─
└─ src/
```

```
├─ components/
│   └─ chat/
│       ├── ChatHistorySidebar.js ← Past sessions sidebar
│       ├── ChatInput.js          ← Message input bar (27KB)
│       ├── ChatMessage.js        ← Single message bubble
│       └─ NotificationBanner.js ← In-chat alert banner
│
│   └─ common/
│       ├── AnimatedButton.js     ← Button with press animation
│       ├── Avatar.js             ← Profile photo / initials
│       ├── Button.js             ← Standard button (variants)
│       ├── Card.js               ← Generic card container
│       ├── EmptyState.js         ← No-data placeholder
│       ├── ErrorState.js         ← Error + retry UI
│       ├── FullScreenSpinner.js  ← Full-screen loader overlay
│       ├── Input.js              ← Styled text input
│       ├── Loader.js             ← Inline spinner
│       ├── LocationCapture.js     ← GPS capture with map preview
│       ├── OfflineBanner.js      ← No-internet alert banner
│       ├── PhotoUploader.js      ← Camera / gallery picker
│       ├── Shimmer.js            ← Shimmer animation primitive
│       ├── SkeletonLoaders.js    ← Full skeleton screens
│       ├── StatusBadge.js        ← Color-coded status pill
│       └─ Toast.js               ← Auto-dismiss notification pop-up
│
│   └─ dashboard/
│       ├── DashboardCard.js      ← Metric summary tile
│       └─ ChartDownloadButton.js ← Export chart as PDF
│
│   └─ issue/
│       ├── IssueCard.js          ← Issue list item (11KB)
│       ├── IssueTimeline.js      ← Issue status history timeline
│       ├── ImageGallery.js       ← Photo gallery viewer
│       └─ CallHistorySection.js ← Call log display
│
│   └─ modals/
│       └─ FilterModal.js         ← Filter/sort bottom sheet (15KB)
│
│   └─ skeletons/
│       ├── DashboardCardSkeleton.js ← Dashboard tile placeholder
│       └─ IssueCardSkeleton.js      ← Issue list item placeholder
│
├─ hooks/
│   ├── useDebounce.js           ← Delay rapidly changing values
│   ├── useNetworkStatus.js      ← Real-time connectivity monitor
│   └─ useSmartBack.js           ← Smart back navigation handler
│
├─ services/
│   ├── api.js                   ← All backend API calls (24KB)
│   └─ imagekitService.js        ← ImageKit CDN upload service
│
├─ store/
│   ├── index.js                 ← Redux store setup
│   └─ slices/
│       ├── authSlice.js         ← Auth token & user state
│       └─ chatSlice.js          ← Chat sessions & messages
```


- └─ complaintsSlice.js ← Complaints data
- └─ dashboardSlice.js ← Dashboard metrics
- └─ issuesSlice.js ← Issues data (17KB)
- └─ notificationsSlice.js ← Notification count & state
- └─ offlineSlice.js ← Offline queue & status
- └─ performanceSlice.js ← Solver performance data
- └─ sitesSlice.js ← Sites data
- └─ themeSlice.js ← Light/dark mode preference
- └─ theme/
 - └─ colors.js ← Full color palette (light + dark)
 - └─ typography.js ← Font sizes, weights, line heights
 - └─ ThemeContext.js ← React context for theme access
- └─ utils/
 - └─ animations.js ← Reusable animation presets
 - └─ constants.js ← App-wide constant values
 - └─ formatters.js ← Date, text formatting helpers
 - └─ haptics.js ← Device vibration feedback
 - └─ location.js ← Basic GPS helpers
 - └─ locationCapture.js ← Advanced location capture logic
 - └─ network.js ← Network status monitoring
 - └─ networkRetry.js ← Exponential backoff retry logic
 - └─ notificationNavigation.js ← Push notification deep-linking
 - └─ overdue.js ← Issue overdue detection
 - └─ pdfExport.js ← PDF generation utility
 - └─ permissions.js ← Device permission management
 - └─ photoUpload.js ← Image compression & upload
 - └─ scoreEngine.js ← Solver performance scoring
 - └─ storage.js ← AsyncStorage wrapper
 - └─ voiceUtil.native.js ← Native voice input (iOS/Android)
 - └─ voiceUtils.js ← Cross-platform voice utilities
- └─ mocks/
 - └─ apiService.js ← Mock API (dev/testing)
 - └─ issues.js ← Mock issue data (12KB)
 - └─ complaints.js ← Mock complaint data
 - └─ chatMessages.js ← Mock chat conversations
 - └─ callLogs.js ← Mock call history
 - └─ issueAssignments.js ← Mock assignment records
 - └─ issueHistory.js ← Mock status timeline events
 - └─ issueImages.js ← Mock image URLs
 - └─ sites.js ← Mock site records
 - └─ solverSkills.js ← Mock solver skill tags
 - └─ supervisorSites.js ← Mock supervisor assignments
 - └─ users.js ← Mock user accounts
- └─ assets/
 - └─ fonts/
 - └─ SpaceMono-Regular.ttf ← Bundled monospace font
 - └─ images/
 - └─ (app icon, splash, illustrations, placeholders)
- └─ scripts/
 - └─ (project reset, asset generation, env setup scripts)

